

FreeBridge 통합 테스트 결과서

문서정보

- 프로젝트명: FreeBridge
- 프론트 서비스명: FreelanceHub B2B CRM
- 문서명: FreeBridge 통합 테스트 결과서
- 최종 작성일: 2026-03-18
- 작성자: 임재열
- 테스트 기간: 2026-03-17 ~ 2026-03-18
- 테스트 환경: AWS ap-northeast-2 / Jenkins / Docker Hub / Kubernetes(EKS)
- 테스트 기준 브랜치: dev
- 제출 형식: PDF
- 관련 저장소: FE / BE / Manifest Repo

1.버전관리

- 별도의 서비스 버전 체계는 운영하지 않았으며, Docker 이미지 업데이트 시 Jenkins Build Number와 Git Short SHA를 조합한 태그를 기준으로 배포 버전을 관리하였다. (예시)

 backend-deployment.yml	[Jenkins] Update backend & AI image to 203-33248b8	9 minutes ago
--	--	---------------

- Spring Application Name: freebridge
- 배포 이미지 태그 정책: Jenkins Build Number
- 기준 프론트 이미지 태그: o2ppo/freebrfront001:53-71cbaea
- 기준 백엔드 이미지 태그: o2ppo/freebrback001:159-e6d94ec
- 기준 AI 이미지 태그: o2ppo/freebridge-ai:159-e6d94ec

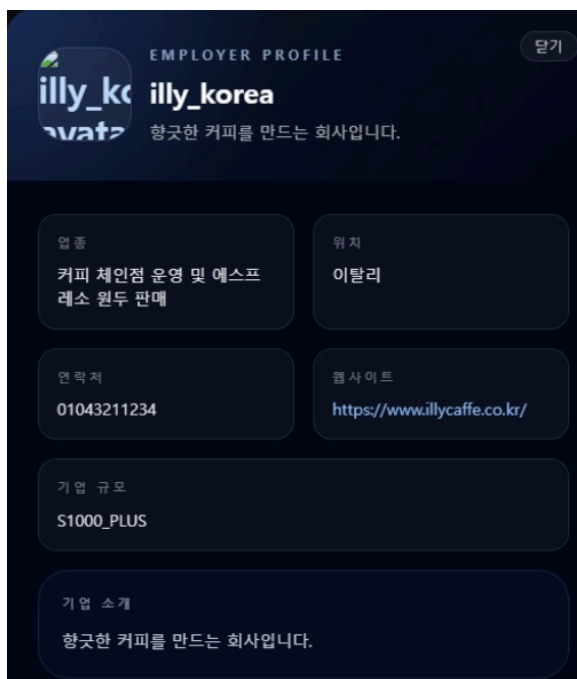
FreeBridge는 프론트엔드, 백엔드, AI 서비스를 분리하여 운영하는 구조이며, 사용자 화면과 API 문서, 내부 배포 버전 체계를 분리해서 관리한다.

2.CI/CD 배포흐름

- Git Commit & Push → GitHub Webhook → Jenkins 파이프라인 트리거 -> EKS 사전 점검(Preflight) -> 소스 Checkout → 애플리케이션 빌드 -> Docker 이미지 생성 및 태깅 -> Docker Hub Push → Manifest Repo 이미지 태그 갱신 -> kubectl apply를 통한 EKS 반영 -> rollout status 확인 -> 배포 완료
 - Jenkins 트리거 이후에 일어나는 일은 모두 젠킨스 파이프라인에서 작동합니다.

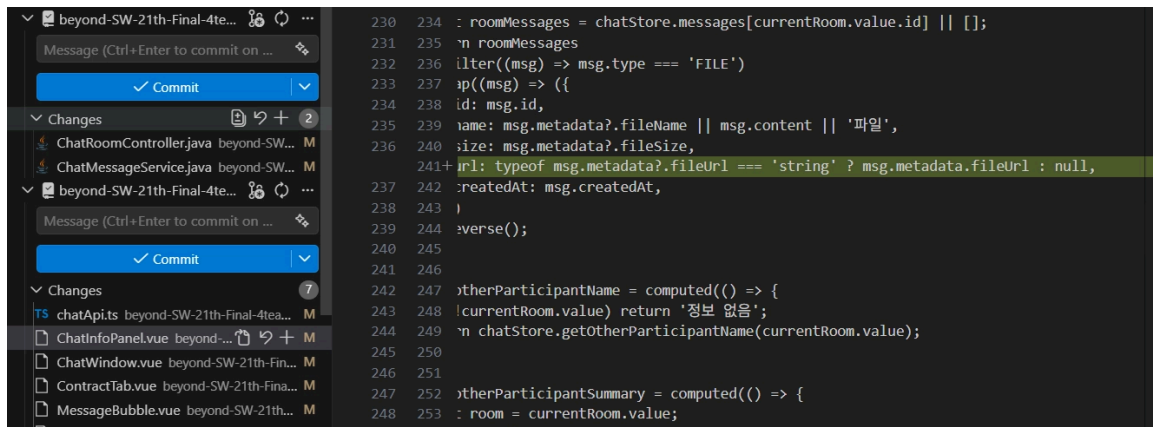
- **FrontEnd와 BackEnd의 CI/CD 흐름은 전체 구조는 유사하나, 백엔드는 Gradle 빌드와 AI 이미지 배포 단계가 추가된다.**

1.변경 사항 반영 전 FreeBridge Service

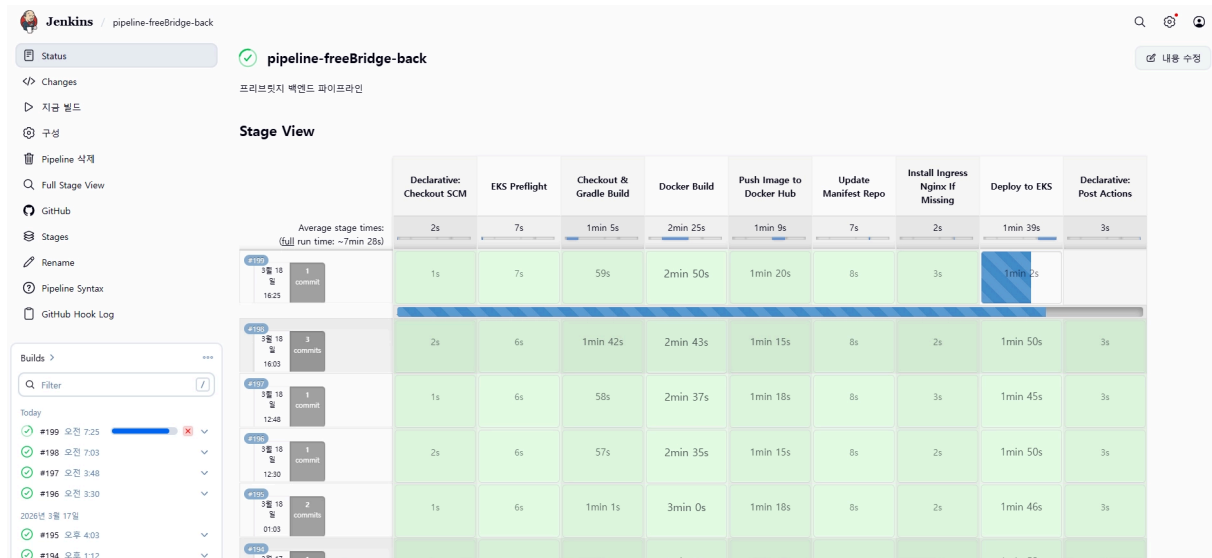


- 프로필 이미지가 보이지 않는 모습이다.

2. 변경 사항 Commit and Push



3. 젠킨스 변경점 감지 → 파이프 라인 실행 → 배포시작



4. Docker 이미지 생성 및 Push 및 태깅 완료

o2ppo
Docker Personal

Repositories

Hardened Images

Collaborations

Settings

Default privacy

Notifications

Billing

Usage

Pulls

Storage

Repositories / freebrback001 / General

o2ppo/freebrback001

Last pushed 29 minutes ago · Repository size: 22.8 GB · ☆0 · ↓2.3K

Add a description

Add a category

General

Tags

Image Management

Collaborators

Webhooks

Settings

Tags

This repository contains 129 tag(s).

Tag	OS	Type	Pulled	Pushed
latest		Image	less than 1 day	29 minutes
203-33248b8		Image	less than 1 day	30 minutes
202-33248b8		Image	less than 1 day	about 1 hour

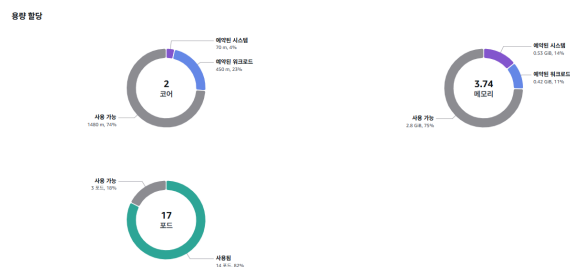
5. 메니페스트 레포지토리 이미지 태그 갱신

backend-deployment.yml

[Jenkins] Update backend & AI image to 203-33248b8

9 minutes ago

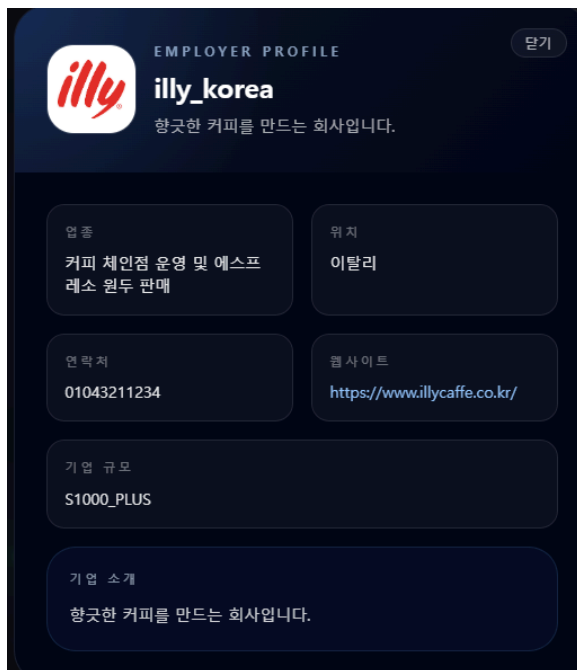
6. AWS EKS에 빌드 확인



포드 (14) 정보

이름	상태	생성됨	IP
backend-6f4799fd6-vdw25	① 실행 중	한 시간 전	10.0.0.4
frontend-9fc95695f-m6wsc	① 실행 중	22분 전	10.0.15.123
ingress-nginx-controller-579c674b5d-6kqlf	① 실행 중	2026년 3월 11일, 01:07 (UTC+09:00)	10.0.6.55
aws-node-dm2r6	① 실행 중	2026년 3월 10일, 15:31 (UTC+09:00)	10.0.12.51
coredns-85fbb8d8d8-95kv7	① 실행 중	2026년 3월 10일, 14:18 (UTC+09:00)	10.0.12.182
coredns-85fbb8d8d8-lhscx	① 실행 중	2026년 3월 10일, 14:18 (UTC+09:00)	10.0.11.18
eks-pod-identity-agent-dfm7x	① 실행 중	2026년 3월 10일, 15:31 (UTC+09:00)	10.0.12.51
kube-proxy-hpspt	① 실행 중	2026년 3월 10일, 15:31 (UTC+09:00)	10.0.12.51
alertmanager-kube-prometheus-stack-alertmanager-0	① 실행 중	2026년 3월 12일, 23:43 (UTC+09:00)	10.0.7.137
kube-prometheus-stack-grafana-78fd4df7c9-xxfrx	① 실행 중	2026년 3월 12일, 23:43 (UTC+09:00)	10.0.9.163

7. 배포 사이트로 접속이후 변경점 확인



- 기존에 프로필이 들어가지 않는 문제가 해결된 모습이다.

8. 모니터링



- 배포 후 주요 모니터링 지표(Backend Up, 5xx Error Rate, P95 Latency, CPU Usage)가 안정적으로 유지되어 서비스가 정상 상태로 운영되고 있음을 확인하였다.

3.전체 인프라 및 환경 요약

- AWS Region: ap-northeast-2
- 컨테이너 오케스트레이션: Amazon EKS freebridge-eks
- CI/CD 엔진: Jenkins
- Jenkins 트리거 방식: githubPush()
- 이미지 저장소: Docker Hub
- Manifest 관리 방식: 별도 Manifest Repository에서 이미지 태그 자동 치환 후 push
- Ingress: ingress-nginx
- 주요 라우팅: / -> Frontend / /api -> Backend / /ws/chat -> Backend
- 오브젝트 스토리지: Amazon S3 freebr-s3
- 관계형 DB: AWS RDS MariaDB
- 모니터링: Prometheus + Grafana
- 백엔드 메트릭 수집 경로: /actuator/prometheus

3-2. 애플리케이션 배포 구성

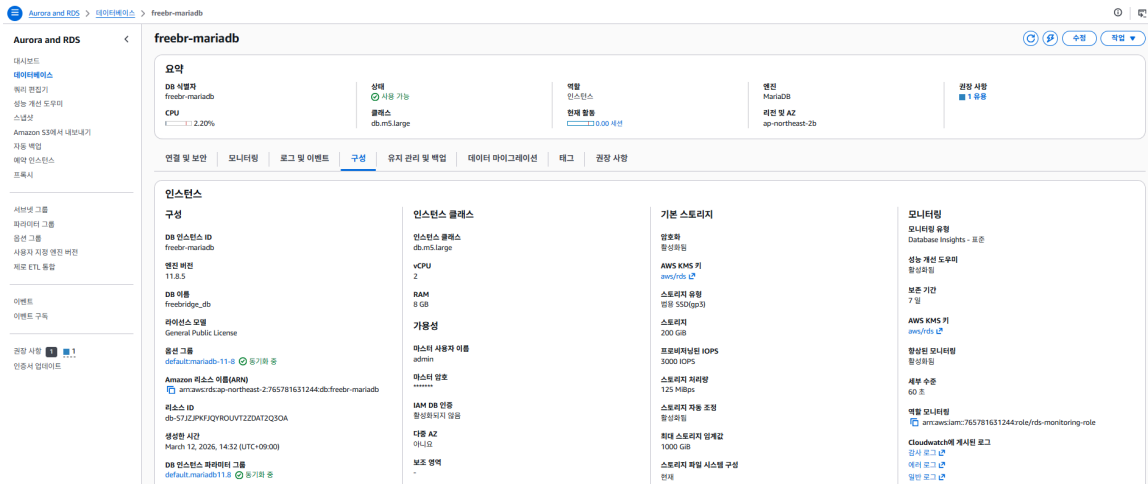
- Front End Deployment: frontend
- Front End Replica: 2
- Front End Service: frontend-service
- Front End Service Port: 80 → 8080
- Back End Deployment: backend
- Back End Replica: 2
- Back End Service: backend-service
- Back End Service Port: 80 → 8080
- Back End Health Check: /actuator/health/liveness, /actuator/health/readiness
- AI Deployment: python-ai-service
- AI Replica: 1
- AI Service: python-ai-service
- AI Service Port: 4009 → 8000

3-3. 실제 라우팅 구성

- / : frontend-service 연결
- /api : backend-service 연결
- /ws/chat : backend-service 연결
- /grafana : kube-prometheus-stack-grafana 연결

3-4. 데이터 및 외부 연동 구성

- MariaDB: AWS RDS MariaDB 사용



- MongoDB: EC2를 사용한 PC 기반 외부 연동
- Redis: EC2를 사용한 PC 기반 Redis 연동
- Python AI: EKS 내부 서비스 호출
- Chroma: EC2를 사용한 PC 기반 기반 외부 연동

3-5. 배포 흐름 요약

GitHub Push → Jenkins Trigger → Docker Build → Docker Hub Push → Manifest Repo Update → kubectl apply → EKS Rollout → Ingress/Service 확인 -> 브라우저/API 검증

4. 통합 테스트 개요

- 통합 테스트 결과는 다음과 같다.

FallGuys-FreeBridge - 통합 테스트 결과 보고서		
프로젝트 명	FreeBridge	
시작일	2026.01.22	
종료일	2026.03.20	
작성자	정재우, 임재열, 이용우, 이형욱, 윤홍석	
1. 단위 테스트 범위 및 결과 요약		
1.1 전체 테스트 현황	UI / UX 테스트	API 테스트
- 총 테스트 케이스 : 235건 - 성공 : 235건 - 실패 : 0건 - 성공률 : 100%	- 전체 : 91/91 성공(100%) - 채팅 : 6/6 성공(100%) - 계약 : 6/6 성공(100%) - 이메일 : 2/2 성공(100%) - 매칭 : 10/10 성공(100%) - 결제 : 12/12 성공(100%) - 광고 : 8/8 성공(100%) - 리뷰 : 9/9 성공(100%) - 유저 : 38/38 성공(100%)	- 전체 : 144/144 성공(100%) - 채팅 : 8/8 성공(100%) - 계약 : 8/8 성공(100%) - 이메일 : 3/3 성공(100%) - 매칭 : 14/14 성공(100%) - 결제 : 28/28 성공(100%) - 광고 : 12/12 성공(100%) - 리뷰 : 13/13 성공(100%) - 유저 : 58/ 58 성공(100%)
2. 주요 테스트 성공 사례		
2.1 매칭	2.2 계약	2.3 후기
- AI 기반 추천 프로세스 구현 완료 - 양방향 매칭 프로세스 구현 완료 - 매칭 상태 변경 시스템 프로세스 구현 완료 - 동시 지원 방지 로직 구현 완료	- 전자 계약 프로세스 및 PDF 자동 생성 구현 완료 - 전자 서명 패드 조작 및 데이터 매핑 성공 - 계약 체결 시 동시성 제어 구현 완료 - 계약 확정 시 관련 광고 자동 마감 시스템 완료	- 상호 평점 및 리뷰 시스템 구현 완료 - AI 평판 분석 보고서 생성 성공 - 리뷰 기반 등급 선정 엔진 구현 완료
3. 개선 항목		
3.1 긴급 개선 필요 항목	3.2 중기 개선 항목	3.3 장기 개선 항목
- 프리랜서 ID 와 USER ID 혼동 해결 - 프리랜서가 같은 기업과 계약 여러개 맺으면 충돌 - 설명에 255자 제한 해제	- 프론트 화이트 모드 테마 추가 - 기업 회원의 헤더 정리 - 기업 요금제의 정기결제 추가	- 이메일을 이용한 진행 사항 알림 전송 - 캐시 처리 시점 변경
4. 결론 및 제언		
전반적으로 기능 구현은 완료되었으며, 특히 매칭에서 계약, 정산,후기 까지의 흐름은 높은 완성도를 보이고 있습니다. 다만, 일부 구현이 되었으나 버그가 발생하거나, UX가 불편한 경우가 발견되어 수정이 필요합니다.		
우선 조치 사항		
1. 프리랜서 ID 와 USER ID 가 혼동되는 경우가 있습니다 이를 해소해야 합니다. 2. 프리랜서가 같은 기업과 계약을 여러개 맺으면 정상적으로 출력이 안될때가 있어 해결해야 합니다. 3. 광고 설명이 255자 제한이 있어 해결해야 합니다.		
프로젝트의 전체적인 완성도는 기능은 완성되었으나 UX 및 자잘한 오류가 있어, 이것들만 개선된다면 안정적이고 사용자가 편리한 서비스가 될 것 입니다.		

4-1.테스트 과정

? 모든 테스트는 아래와 같은 과정을 진행하였다.

- 프론트엔드, 백엔드, AI 서버, 데이터 계층까지 전체 배포 흐름이 정상 동작하는지 검증
- Jenkins 기반 CI/CD 파이프라인이 실제 운영 구성과 연결되어 있는지 확인
- Docker Hub 이미지 갱신, Manifest Repo 반영, EKS Rollout, Ingress 라우팅, 최종 서비스 반영까지 전 구간 증빙이후 배포환경에서 테스트가 성공하는지.

4-2. 범위

- Front End: GitHub Push, Jenkins Trigger, Docker Build/Push, Manifest 반영, FE Rollout, 브라우저 반영
- Back End: Gradle Build, Docker Build/Push, Manifest 반영, EKS Deploy, Health Check, API 반영
- AI: AI 이미지 빌드/배포, Python AI Service 기동, 백엔드 연동 결과 확인
- Data/Infra: RDS, Redis, MongoDB, Chroma, Ingress, Monitoring 확인

4-3. 판정 기준

- Jenkins: 전체 Stage 성공 및 실패 없이 종료
- Docker Hub: 신규 태그 이미지 업로드 확인
- Manifest Repo: 신규 태그가 실제 YAML에 반영되고 push 완료
- EKS Rollout: 각 Deployment 정상 rollout 완료
- 서비스 반영: 브라우저 또는 API에서 수정 사항 확인
- 데이터 연동: DB, Redis, AI, 관련 오류 없이 기능 수행 했는가?

4-4. 사전 준비 항목

1. GitHub 변경 사항 준비
2. Jenkins 접속 가능 여부 확인
3. Docker Hub 접속 가능 여부 확인
4. AWS 콘솔 또는 kubectl 확인 가능 여부 확보
5. Postman 또는 실제 서비스 검증 URL 확보

5.모니터링



5-1. 실제 구성 정보

- Grafana 경로: /grafana
- Grafana 서비스: kube-prometheus-stack-grafana
- Prometheus 보존 기간: 15d
- Prometheus 수집 주기: 30s
- Backend Metrics Path: /actuator/prometheus
- Storage Class: gp3

5-2. 중점 모니터링 지표와 확인 이유

- Backend Up: 현재 정상 응답 가능한 백엔드 인스턴스 수를 의미한다. 배포 직후 Replica 수가 기대값인 2를 유지하는지 확인하는 가장 직접적인 지표이며, 값이 줄어든다면 Pod 장애, rollout 실패, readiness 실패를 빠르게 파악할 수 있다.
- 5xx Error Rate: 서버 내부 오류 발생량을 의미한다. 사용자가 실제로 장애를 체감하는 가장 직접적인 신호라서, 배포 직후 기능 회귀나 예외 폭증 여부를 우선 판단할 때 본다.
- Traffic RPS: 초당 요청 수를 의미한다. 현재 트래픽 수준을 알아야 latency, CPU, thread 사용량을 해석할 수 있으므로 기준 부하를 확인하는 용도로 중요하다.
- P95 Latency: 전체 요청 중 느린 측에 속하는 95% 구간의 응답 시간을 의미한다. 평균값보다 사용자 체감 성능 저하를 더 잘 드러내므로, 성능 악화를 조기에 확인할 때 본다.
- JVM Heap: 자바 애플리케이션 메모리 사용량을 의미한다. 사용량이 지속적으로 우상향하면 메모리 누수나 GC 압박, OOM 위험 신호일 수 있어 장기 안정성을 보기 위해 확인한다.

- Tomcat Threads: 요청 처리 스레드 사용 현황을 의미한다. busy threads가 current threads 또는 최대치에 가까워지면 요청 적체, 지연, 타임아웃 가능성이 높아진다.
- HikariCP: 데이터베이스 커넥션 풀 상태를 의미한다. active가 max에 근접하거나 pending이 증가하면 DB 병목 또는 커넥션 풀 부족 가능성을 의심할 수 있다.
- CPU Usage: 프로세스 및 시스템 CPU 사용량을 의미한다. 배포 직후 비정상 로직, 무한 루프, 과도한 부하가 발생하는지 판단하는 기본 자원 지표다.

5-3. 현재 화면 기준 해석 예시

- Backend Up = 2: 현재 백엔드 Replica 2개가 모두 정상 동작 중이라는 의미다.
- 5xx Error Rate = 0 req/s: 현재 시점에서 서버 내부 오류가 관측되지 않았음을 의미한다.
- Traffic RPS = 0.370 req/s: 낮지만 지속적인 요청이 유입되고 있어 서비스가 실제 호출되고 있음을 보여준다.
- P95 Latency = 12.2 ms: 현재 응답 지연이 낮은 수준으로 유지되고 있어 사용자 체감 성능이 안정적이라는 해석이 가능하다.
- JVM Heap: 사용량이 급격히 증가하지 않고 안정적으로 유지되고 있어 메모리 누수 징후는 즉시 보이지 않는다.
- Tomcat Threads: 바쁜 스레드 수가 매우 낮고 현재 스레드 여유가 충분해 요청 처리 병목이 보이지 않는다.
- HikariCP: 활성 커넥션과 대기 요청이 낮아 DB 커넥션 풀이 포화 상태가 아님을 보여준다.
- CPU Usage: 프로세스 CPU와 시스템 CPU 모두 낮은 수준에서 안정적으로 유지되고 있어 현재 부하 여유가 있는 상태로 볼 수 있다.